

Deep-Research Appendix (cited external findings)

Deep-Research Appendix (cited external findings)

Revision: 4 Last modified: 2026-07-04T12:00:00Z Rev 4 (lighter-touch consistency audit, §11.4.99/§11.4.150): Cross-checked this appendix's ten per-angle summaries (§A1–§A10) against the full text of every v09-research/research-*.md dossier + _SYNTHESIS.md. Result: **no factual contradictions found** — RFC numbers (9298/9297/9221/9220/9484), WireGuard wire sizes (148/92/64/32-byte overhead), MTU arithmetic (1420/1280), the maybenot/DAITA v1↔v2 constants, and the Rust-vs-Go tooling-maturity verdicts (MASQUE: no turnkey Rust crate vs masque-go turnkey; Go control-plane stack versions) are stated identically wherever they recur across files. All ten per-angle [DONE][RES] statuses in MASTER_INDEX.md are confirmed accurate — none overstate or understate what its dossier actually contains. The one consistency gap found — 9 of 10 per-angle dossiers (all but research-wireguard.md) lacked the §11.4.44 revision header — is fixed in this same pass (each now carries Revision: 1, _SYNTHESIS.md included).

A0. Purpose, provenance, and verification honesty (§11.4.6 / §11.4.99 / §11.4.123)

This appendix collects the **external, citable findings** that shaped the HelixVPN master specification (docs 00–10 in this final/ set). It is organised as **one section per research angle**. Each section gives: **(1)** the key findings that drove a spec decision, **(2)** concrete version pins / API facts / constraints discovered, and **(3)** a **Sources** list (canonical URLs + access date) copied verbatim from that angle's research file.

A0.1 Provenance — read this first

This revision is rebuilt from the now-collected per-angle research corpus under v09-research/research-*.md, each entry produced 2026-06-25 with live web access. **All ten planned angles are now verified and cited** below — the tenth angle, **WireGuard core protocol (§A1)**, was re-run 2026-06-25 with live web access and is backed by v09-research/research-wireguard.md (511 lines, official wireguard.com protocol page + messages.h constants + cloudflare/boringtun + docs.rs Tunn API + wireguard-nt). The prior "UNVERIFIED — research file missing" marker is **resolved**; all "UNVERIFIED (no web access)" markers across the corpus are replaced with cited facts.

§	Angle	Status
A1	WireGuard core protocol	Verified 2026-06-25
A2	MASQUE / QUIC obfuscation (masque)	Verified 2026-06-25
A3	Hysteria2 / Salamander / censorship regimes (hysteria2)	Verified 2026-06-25
A4	Mullvad full-feature parity (mullvad)	Verified 2026-06-25
A5	Flutter↔Rust FFI (flutter_ffi)	Verified 2026-06-25
A6	iOS/Android native tunnel core (ios_android)	Verified 2026-06-25
A7	Go control plane (go_cp)	Verified 2026-06-25
A8	Podman/Compose/K8s deployment (podman_k8s)	Verified 2026-06-25
A9	PKI / post-quantum / NAT traversal (pki_pq_nat)	Verified 2026-06-25
A10	DAITA / maybenot / test-rig methodology (daita_test)	Verified 2026-06-25

A1. WireGuard core protocol (Noise IK + boringtun/kernel/wireguard-go/wireguard-nt)

Status (§11.4.6): Verified 2026-06-25 with live web access; full dossier at v09-research/research-wireguard.md. The previously-missing angle is re-run against **official primary sources** — the official Protocol & Cryptography page, the official messages.h

constants header, cloudflare/boringtun, docs.rs Tunn API, and the wireguard-nt about page. **Honest gap:** the formal whitepaper PDF (wireguard.com/papers/wireguard.pdf) was **UNREACHABLE** via the fetch tool (returned FlateDecode binary, not text) — every value it canonically sources was re-confirmed from a different authoritative source, so no claim rests on the unreachable PDF.

Key findings that shape the spec (all cited in the dossier).

- **Noise_IK, 1-RTT** handshake; the initiator knows the responder's static pubkey in advance; the responder cannot send data until key confirmation and never initiates a rekey (Noise_IKpsk2 when the PSK is in use) [wg-protocol <https://www.wireguard.com/protocol/>].
- **Crypto set (CONFIRMED):** Curve25519 (X25519), ChaCha20Poly1305 (RFC 7539, 64-bit counter nonce), BLAKE2s + HKDF, **SipHash24** (index hashtable), **XChaCha20Poly1305** (cookie), TAI64N (12-byte handshake-replay timestamp) [wg-protocol].
- **Message sizes (CONFIRMED, were a primary fingerprint):** Handshake Initiation **148 B**, Handshake Response **92 B**, Cookie Reply **64 B**, Transport Data = 16-byte header + AEAD (16-byte data hdr + 16-byte Poly1305 tag = **32-byte WG overhead**) [wg-protocol].
- **Timers (CONFIRMED from messages.h):**
REKEY_AFTER_TIME=120 s, REKEY_TIMEOUT=5 s,
REKEY_ATTEMPT_TIME=90 s (→ MAX_TIMER_HANDSHAKES=18),
REJECT_AFTER_TIME=180 s, KEEPALIVE_TIMEOUT=10 s,
REKEY_AFTER_MESSAGES=2^60, REJECT_AFTER_MESSAGES≈2^64,
COOKIE_SECRET_MAX_AGE=120 s, replay window
COUNTER_BITS_TOTAL=2048 [wg-messages-h <https://github.com/WireGuard/wireguard-monolithic-historical/blob/master/src/messages.h>].
- **MTU 1420 (CONFIRMED):** 1500 – 60 B IPv4 (20 IP + 8 UDP + 32 WG) / – 80 B IPv6 overhead; 1280 is the robust mobile floor (IPv6 min-MTU) [wg-mtu].
- **PSK = the post-quantum injection point (CONFIRMED):** the optional 32-byte PSK (all-zero when unused) is mixed into the symmetric key schedule (temp = HMAC(responder.chaining_key, preshared_key)), so the tunnel survives a broken X25519 — exactly the slot Mullvad's PQ KEM secret fills (§A9, §A4) [wg-protocol].
- **AllowedIPs = Cryptokey Routing (CONFIRMED):** dest-IP → peer for outbound, source-IP ∈ peer's AllowedIPs for inbound default-deny; UAPI keys public_key/allowed_ip/preshared_key/endpoint/persistent_keepalive_interval [wg-xplatform <https://www.wireguard.com/xplatform/>].
- **boringtun (CONFIRMED):** Cloudflare pure-Rust userspace WG, BSD-3-Clause, deployed on "millions of iOS and Android consumer devices ... thousands of Cloudflare Linux servers" (1.1.1.1/WARP). **Maintenance note (§11.4.6): master branch explicitly unstable — "you should probably not**

rely on or link to the master branch right now"; use crates.io releases. The sans-IO Tunn API is CONFIRMED:
 Tunn::new(static_private, peer_static_public, preshared_key: Option<[u8;32]>, persistent_keepalive, index, rate_limiter), encapsulate(src, dst), decapsulate(src_addr, datagram, dst), update_timers(dst) → TunnResult
 {Done,Err,WriteToNetwork,WriteToTunnelV4,WriteToTunnelV6} — the caller-provided dst: &mut [u8] matches helix-wg's zero-alloc-hot-path invariant W1 [boringtun-gh <https://github.com/cloudflare/boringtun>; boringtun-tunn <https://docs.rs/boringtun/latest/boringtun/noise/struct.Tunn.html>].

- **Backends (CONFIRMED):** kernel WG (mainline since Linux 5.6, fastest, plain-udp only), boringtun (cross-platform floor incl. iOS where no kernel WG exists), **wireguard-go** (official Go userspace reference; the base Mullvad DAITA forks), **wireguard-nt** ("High performance **in-kernel** WireGuard implementation for Windows", Win10/11 AMD64/x86/ARM64, ships as wireguard.dll — "the only official and recommended way of using WireGuard on Windows") [wg-nt <https://git.zx2c4.com/wireguard-nt/about/>; wg-xplatform].
- **Why HelixVPN wraps it:** WireGuard has **no built-in obfuscation** (DPI-fingerprintable, UDP-blockable), leaks size/timing side-channels, rekeys every ~2 min + handshakes on first packet, ships the PSK empty, and has no PKI/coordination — each gap maps to a HelixVPN layer (helix-transport / DAITA / PQ-PSK / control plane).

Cross-references retained: the 148/92-byte fingerprint + UDP-only limitation also appear under §A3 (AmneziaWG), the Curve25519-pubkey-as-identity + PSK-mixing model under §A9, and the wireguard-go/maybenot integration path under §A4/§A10 — now backed by the primary WireGuard sources here.

Sources (accessed 2026-06-25; full list in the dossier):

- wg-protocol — <https://www.wireguard.com/protocol/> (REACHED, primary)
- wg-paper — <https://www.wireguard.com/papers/wireguard.pdf> (**UNREACHABLE:** FlateDecode binary, not parseable text — values re-confirmed elsewhere)
- wg-xplatform — <https://www.wireguard.com/xplatform/>
- wg-messages-h — <https://github.com/WireGuard/wireguard-monolithic-historical/blob/master/src/messages.h>
- boringtun-gh — <https://github.com/cloudflare/boringtun>
- boringtun-noise — <https://docs.rs/boringtun/latest/boringtun/noise/index.html>
- boringtun-tunn — <https://docs.rs/boringtun/latest/boringtun/noise/struct.Tunn.html>
- wg-nt — <https://git.zx2c4.com/wireguard-nt/about/>
- wg-mtu / wg-timers — WireGuard MTU + timer constants corroborated by wireguard-go device/timers.go, torvalds/linux

A2. MASQUE / QUIC obfuscation (RFC 9298 CONNECT-UDP over HTTP/3)

Key findings that shape the spec.

- WG-over-MASQUE is a layering of four **published Proposed-Standard** IETF RFCs (stable, not drafts): **RFC 9298** (Proxying UDP in HTTP / CONNECT-UDP, the control plane), **RFC 9297** (HTTP Datagrams + Capsule Protocol + Quarter-Stream-ID mux), **RFC 9221** (QUIC DATAGRAM frame — unreliable, negotiated via `max_datagram_frame_size`), and **RFC 9220** (Extended CONNECT over HTTP/3, supplies the `:protocol` pseudo-header). **RFC 9484** (CONNECT-IP) is the full-L3-tunnel sibling — not what Mullvad uses for WG obfuscation but relevant if Helix later wants a full L3 MASQUE tunnel.
- **Data plane:** one WG UDP packet → one HTTP Datagram (9297) → one QUIC DATAGRAM frame (9221) tagged with the CONNECT-UDP stream's Quarter-Stream-ID → proxy (9298) emits a plain UDP packet to the WG server. Spec **MUST** use QUIC DATAGRAM (never the reliable stream) to preserve WG's loss-tolerant semantics and avoid head-of-line blocking.
- **Mullvad parity target:** QUIC obfuscation shipped on desktop in **2025.9 (Sept 2025)** and on Android/iOS in **2025.8+**. It is built directly on MASQUE/RFC 9298 over HTTP/3 on UDP/443; the proxy runs **server-side on Mullvad relays**; later builds randomly select one of several in-addresses per connection attempt. **Implementation = Rust**; strong evidence the QUIC stack is **quinn** (changelog references the `quinn_udp` crate). Default behaviour: auto-try QUIC after failed normal/obfuscation attempts; forcible via `mullvad obfuscation set mode quic`.
- **Throughput cost is real and Mullvad-acknowledged** ("computationally very expensive ... affects throughput") — the double-crypto + double-congestion-control tax. The spec **MUST** require captured throughput evidence WG-direct vs WG-over-MASQUE and **MUST NOT** claim parity speed.
- **DPI/fingerprint is the hard, unsolved part.** Mullvad's public story is collateral-damage deterrence (blocking HTTP/3-on-443 risks breaking the open web). The exact TLS/QUIC ClientHello fingerprint mimicry (uTLS-style), SNI, ALPN, domain-fronting are **NOT publicly documented** — treat QUIC/TLS fingerprint mimicry as an explicit open design/risk item, not a solved checkbox.

Version pins / API facts / constraints.

- **Rust: NO turnkey CONNECT-UDP crate.** Hand-roll on quinn (mature async QUIC, rustls TLS 1.3, quinn-proto sans-I/O, DATAGRAM/RFC 9221 support) + h3 (hyperium/h3, generic over transport via h3-quinn). The CONNECT-UDP pieces are experimental and split: **h3-datagram v0.0.2** (RFC 9297, pre-1.0 "API subject to change"), Extended-CONNECT as a separate modular feature, h3-webtransport ("API subject to change ... may contain bugs ... not yet complete"). Third-party jromwu/masquerade is research-grade, not a maintained library. tokio-quiche (Cloudflare, open-sourced Dec 2025, over quiche) is a newer base, still no turnkey CONNECT-UDP helper. Budget the Rust path as real engineering.
- **Go: turnkey library exists. quic-go/masque-go v0.3.0 (2025-06-24)**, MIT, actively maintained (tracks latest two Go releases) — provides **both client and proxy** for RFC 9298 over quic-go's DATAGRAM support; ~236★, pre-1.0, basic CONNECT-UDP (no CONNECT-IP advertised). quic-go core is mature (used by Caddy); CONNECT-UDP lives in masque-go, not core.
- **Language trade-off:** Rust+quinn = max fingerprint/perf control (matches Mullvad) but hand-rolled RFC 9298; Go+masque-go = fastest working tunnel but weaker fingerprint control (harder uTLS-equivalent in Go) — good for a first iteration / reference proxy.
- **Reuse before reimplement (§11.4.74):** Go path → adopt quic-go/masque-go; Rust path → adopt quinn+h3/h3-datagram and contribute any CONNECT-UDP helper upstream.

Sources (accessed 2026-06-25):

- RFC 9298 — <https://datatracker.ietf.org/doc/rfc9298/>
- RFC 9297 — <https://datatracker.ietf.org/doc/html/rfc9297>
- RFC 9221 (via summaries) — <https://datatracker.ietf.org/doc/rfc9298/> + <https://quic-go.net/docs/http3/datagrams/>
- Mullvad — Introducing QUIC Obfuscation for WireGuard — <https://mullvad.net/en/blog/introducing-quic-obfuscation-for-wireguard>
- Mullvad — QUIC obfuscation on Android and iOS — <https://mullvad.net/en/blog/quic-obfuscation-now-available-on-android-and-ios>
- CyberInsider — <https://cyberinsider.com/mullvad-adds-quic-obfuscation-for-wireguard-to-evade-censorship/>
- mullvadvpn-app CHANGELOG — <https://github.com/mullvad/mullvadvpn-app/blob/main/CHANGELOG.md>
- mullvadvpn-app repo — <https://github.com/mullvad/mullvadvpn-app>
- quinn — <https://github.com/quinn-rs/quinn> + <https://docs.rs/quinn/latest/quinn/>

- h3 releases + WebTransport/datagram discussion — <https://github.com/hyperium/h3/releases> + <https://github.com/hyperium/h3/discussions/189>
 - h3-webtransport — <https://lib.rs/crates/h3-webtransport>
 - masque-go — <https://github.com/quic-go/masque-go> + <https://pkg.go.dev/github.com/quic-go/masque-go>
 - masque-go releases (v0.3.0, 2025-06-24) — <https://github.com/quic-go/masque-go/releases>
 - quic-go CONNECT-UDP docs — <https://quic-go.net/docs/connect-udp/>
 - quic-go HTTP Datagrams docs — <https://quic-go.net/docs/http3/datagrams/>
 - jromwu/masquerade — <https://github.com/jromwu/masquerade>
 - InfoQ — Cloudflare open-sources tokio-quiche — <https://www.infoq.com/news/2025/12/quic-http3-rust/>
-

A3. Hysteria2 / Salamander / AmneziaWG — censorship-regime survivability

Key findings that shape the spec.

- **Hysteria2** is a TCP/UDP proxy on **QUIC (RFC 9000) + Unreliable Datagram (RFC 9221)**, protocol "v4" since v2.0.0, with a custom **Brutal** congestion control that deliberately ignores packet loss ("bandwidth cheating") to maximise throughput on lossy/throttled links.
- **Salamander obfuscation** encapsulates *all* QUIC packets: BLAKE2b-256 hash of a random 8-byte salt + a user pre-shared key, XOR-scrambling every packet to random bytes — defeats QUIC/HTTP-3 fingerprinting (hides the QUIC Initial + SNI). Does **not** help where UDP itself is blocked. **Gecko (experimental)** additionally fragments the QUIC handshake into random-sized, random-padded chunks (min/max_packet_size), directly defeating the GFW's single-datagram QUIC-Initial inspection.
- **Censorship-regime matrix (the spec's transport-selection logic):**
 - **China GFW QUIC-SNI censorship (USENIX Sec '25):** GFW derives the QUIC-Initial key from the packet header (key is header-derivable per RFC 9001), extracts SNI, matches a blocklist (~43.8K FQDNs/week). **Critical weakness:** it does NOT reassemble Initials split across >1 UDP datagram → Gecko fragmentation / Salamander scramble survive; plain QUIC does not. Enforcement is residual (3-min block, ~500 ms onset).
 - **Hard UDP block:** QUIC/Hysteria2/MASQUE-over-H3/WireGuard/AmneziaWG **all fail** → need TCP camouflage

(VLESS+Reality, ShadowTLS, AnyTLS) or **udp2tcp** (dndx/phantun fake-TCP, 12-byte overhead, user-mode TCP, 100% safe Rust). MASQUE's HTTP/2 fallback is the in-protocol analogue.

- **SNI/TLS DPI on TCP 443:** survivors = VLESS+Reality (real-site TLS fingerprint), ShadowTLS, AnyTLS, or Salamander-scrambled QUIC (no SNI on wire).
- **Active probing:** Hysteria2 password auth + Salamander returns nothing to an unauthenticated prober.
- **AmneziaWG** is the lowest-overhead WG-native DPI evasion: breaks the WG fingerprint (fixed message types 1-4, fixed 148-byte handshake-init / 92-byte response) via junk packets (Jc count, Jmin..Jmax size) and AmneziaWG-2.0 dynamic header ranges + random padding so every server speaks its own dialect. ~3% throughput cost. Still UDP-based → pair with Phantun for UDP-blocked nets.
- **Synthesis:** Mullvad chose MASQUE because it is an IETF standard that genuinely looks like HTTP/3 and has in-protocol HTTP/2 fallback; Hysteria2/Gecko beats it on raw throughput over throttled links and out-of-box GFW-QUIC evasion, with more mature self-host tooling, but is non-standard "random noise." **Neither survives a hard UDP block alone.** Recommended layered design: MASQUE **or** Hysteria2/Gecko QUIC primary → AmneziaWG for low-overhead WG-fingerprint evasion → Phantun + VLESS-Reality/ShadowTLS as UDP-blocked / SNI-filtered fallback chain (mirrors Mullvad's "auto-try after failure" model).

Version pins / API facts / constraints.

- **apernet/hysteria latest = app/v2.9.2 (2026-05-23)** — adds Gecko obfuscation, fixes a UDP ACL-bypass security issue, OOM hardening ("strongly encourage everyone to upgrade").
- **sing-box** supports Hysteria2, TUIC, VLESS+Reality, ShadowTLS, **AnyTLS** (added ~March 2025), Shadowsocks, Trojan, WireGuard; Gecko added as a new QUIC obfs type.
- **WebFetch was blocked** on v2.hysteria.network and mullvad.net; those facts were worked around via mirrors/search summaries (noted per source below).

Sources (accessed 2026-06-25):

- Hysteria2 protocol (via WebSearch summary; direct fetch blocked) — <https://v2.hysteria.network/docs/developers/Protocol/>
- apernet/hysteria releases (app/v2.9.2, 2026-05-23) — <https://github.com/apernet/hysteria/releases>
- Hysteria changelog (via search) — <https://v2.hysteria.network/docs/Changelog/>
- sing-box Hysteria2 — <https://sing-box.sagernet.org/configuration/outbound/hysteria2/>

- sing-box changelog (AnyTLS Mar 2025, Gecko) — <https://sing-box.sagernet.org/changelog/>
 - Mullvad QUIC obfuscation (via mirror; direct fetch blocked) — <https://mullvad.net/en/blog/2025/9/9/introducing-quic-obfuscation-for-wireguard>
 - AlternativeTo — <https://alternativeto.net/news/2025/9/mullvad-vpn-adds-quic-obfuscation-for-wireguard-to-bypass-censorship/>
 - RFC 9298 CONNECT-UDP draft — <https://ietf-wg-masque.github.io/draft-ietf-masque-connect-udp/draft-ietf-masque-connect-udp.html>
 - MASQUE / HTTP-2 fallback — <https://http.dev/masque>
 - masque-go — <https://github.com/quic-go/masque-go>
 - AmneziaWG params — <https://docs.amnezia.org/documentation/amnezia-wg/>
 - AmneziaWG 2.0 (junk packets, 148/92-byte fingerprint, ~3% perf) — <https://dev.to/bivlkd/amneziawg-20-self-host-an-obfuscated-wireguard-vpn-that-bypasses-dpi-4692>
 - amneziawg-go — <https://deepwiki.com/amnezia-vpn/amneziawg-go>
 - USENIX Sec '25 SNI-based QUIC censorship — <https://gfw.report/publications/usenixsecurity25/en/>
 - GFW QUIC-Initial key derivation / residual censorship — <https://upb-syssec.github.io/blog/2025/quic-china/>
 - The Register — GFW QUIC analysis — https://www.theregister.com/2025/08/04/china_great_firewall_quic_security_flaws/
 - GFW Shadowsocks active probing — https://gfw.report/blog/gfw_shadowsocks/
 - Defend against GFW active probing — https://gfw.report/blog/ss_advise/en/
 - Phantun udp2tcp (12-byte overhead, fake-TCP) — <https://github.com/dndx/phantun>
 - Phantun internals — <https://deepwiki.com/dndx/phantun>
 - WireGuard UDP-only limitation — <https://www.wireguard.com/known-limitations/>
-

A4. Mullvad full-feature parity bar

Key findings that shape the spec.

- **DAITA** (Defense Against AI-guided Traffic Analysis) defeats ML website-fingerprinting on packet size+timing via three pillars: **constant packet sizes** (uniform padding), **random bidirectional cover traffic**, and **data-pattern distortion**. Built on **maybenot** with one instance at client + one at relay. **DAITA v2** moved from hardcoded client machines to **server-driven** defenses (relay selects from a DB and pushes machines + padding budgets at connect time), **~50% lower bandwidth** vs v1, rotatable server-side without a client update. Self-host =

a wireguard-go fork that reports per-packet events into maybenot + uniform-MTU padding + a server that ships machine definitions + budgets at handshake (machines are data, not code).

- **Multihop = WireGuard-in-WireGuard**, double-encrypted on the client device; the WG **Endpoint port selects the exit, Endpoint IP/host selects the entry**; entry port-maps to the chosen exit. Trust split: entry sees source IP + chosen exit but not traffic; exit sees traffic but only entry's IP. Self-host needs ≥ 2 meshed servers + a port→exit forwarding map per entry — simpler than a bespoke relay protocol.
- **Quantum-resistant tunnels** = establish vanilla WG → negotiate a PQ shared secret over it → install as WG **PSK** (mixed into the symmetric key schedule). KEM options: cme (Classic McEliece 460896 R3), mlkem (ML-KEM-1024 / FIPS 203), cme-mlkem (**default hybrid**), mlkem-cme. ~1-2 s extra setup, steady-state unchanged. Tool mullvad-upgrade-tunnel via WG PostUp. Reusable: liboqs / Rust ml-kem + classic-mceliece-rust.
- **Four WG obfuscation modes**: udp2tcp (Rust udp-over-tcp crate), Shadowsocks, QUIC/ MASQUE (desktop 2025.9+), and **Lightweight WireGuard Obfuscation (LWO, 2025)** — cheap keyed header-scramble, best throughput/power; Mullvad-specific. Auto/adaptive mode tries them after failed attempts.
- **Encrypted DNS + content blocking**: each server runs its own local resolver (all clients egress behind one IP). Blocking tiers via resolver hostnames: dns (none), adblock, base (+malware), family (+adult), extended (+social), all. DoH + DoT; malware uses URLHaus RPZ; blocklists open (mullvad/dns-blocklists). Self-host: per-server unbound/knot with RPZ + category sub-resolvers.
- **Kill switch is always-on fail-closed firewall rules** (not a process) gated on tunnel state; **lockdown mode** extends fail-closed to the disconnected state; **split tunneling** excludes apps (OS-specific: cgroup/netns on Linux, per-app on Android).
- **Privacy is architectural, not policy**: account = one random number (no username/password/email); payments decoupled (cash/Monero/BTC); **no logging** (audited); sim-connection limit (5) enforced **in RAM only**; **diskless/RAM-only servers** (System Transparency stboot net-boot) so a seized server yields no user data.

Version pins / API facts / constraints.

- **maybenot** = Rust, MIT, maybenot-io/maybenot, v2.x current; peer-reviewed at WPES'23 (doi 10.1145/3603216.3624953); lineage = Tor Circuit Padding Framework → WTF-PAD. Ships maybenot-ffi (C ABI) + a simulator.
- **PQ default/available on all WG servers**; Classic McEliece's ~half-MB public key is why it is used for the static/server key and ML-KEM for the ephemeral part.

- **Honest gaps (§11.4.6):** the DAITA-v2 machine/budget push wire format, the multihop port→exit table format, and the PQ ephemeral-peer message sequence are NOT in public docs.

Sources (accessed 2026-06-25):

- DAITA blog — <https://mullvad.net/en/blog/introducing-defense-against-ai-guided-traffic-analysis-daita>
 - DAITA page — <https://mullvad.net/en/vpn/daita>
 - CyberInsider DAITA v2 — <https://cyberinsider.com/mullvads-daita-v2-brings-stronger-resistance-to-ai-enhanced-vpn-traffic-analysis/>
 - pulls.name DAITA v1/v2 defenses — <https://pulls.name/blog/2025-03-27-daita-v1-and-v2-defenses/>
 - maybenot — <https://github.com/maybenot-io/maybenot> · <https://crates.io/crates/maybenot> · <https://docs.rs/maybenot/latest/maybenot/>
 - maybenot WPES'23 — <https://dl.acm.org/doi/10.1145/3603216.3624953>
 - maybenot overview — <https://www.ethanwitwer.com/posts/maybenot-framework/>
 - Multihop — <https://mullvad.net/en/help/multihop-wireguard> · <https://mullvad.net/en/blog/wireguard-multihop-now-easy-available-app>
 - Quantum-resistant tunnels — <https://mullvad.net/en/help/quantum-resistant-tunnels-with-wireguard> · <https://mullvad.net/en/blog/stable-quantum-resistant-tunnels-in-the-app> · <https://mullvad.net/en/blog/post-quantum-safe-vpn-tunnels-available-on-all-wireguard-servers> · <https://mullvad.net/en/blog/introducing-post-quantum-vpn-mullvads-strategy-future-problem>
 - Obfuscation — <https://mullvad.net/en/blog/introducing-quick-obfuscation-for-wireguard> · <https://mullvad.net/en/blog/introducing-lightweight-wireguard-obfuscation> · <https://mullvad.net/en/blog/introducing-shadowsocks-obfuscation-for-wireguard> · <https://discuss.privacyguides.net/t/how-to-choose-the-best-obfuscation-method-with-mullvadvpn/32227>
 - DNS — <https://mullvad.net/en/blog/adding-another-layer-malware-dns-blocking> · <https://mullvad.net/en/blog/how-were-knocking-down-ads-and-tracking> · <https://mullvad.net/en/help/dns-over-https-and-dns-over-tls> · <https://github.com/mullvad/dns-blocklists>
 - Kill switch / split tunnel — <https://mullvad.net/en/help/using-mullvad-vpn-app> · <https://github.com/mullvad/mullvadvpn-app/blob/main/docs/security.md> · <https://mullvad.net/en/help/split-tunneling-with-the-mullvad-app>
 - No-logging / accounts — <https://mullvad.net/en/help/no-logging-data-policy> · <https://mullvad.net/en/blog/mullvads-account-numbers-get-longer-and-safer> · <https://www.threads.com/@jacaranda7/post/DRzPij5kbA2/>
-

A5. Flutter ↔ Rust FFI (driving the Rust VPN core from Flutter)

Key findings that shape the spec.

- **flutter_rust_bridge (frb) v2 is the recommended bridge.** Codegen (`flutter_rust_bridge_codegen generate`) emits Dart FFI glue + a Rust wrapper from annotated plain Rust. V2 supports arbitrary Rust/Dart types incl. opaque handles (`RustOpaque`), **async Rust async fn** (natural for a tunnel-supervisor task), **Rust→Dart calls** (e.g. ask Dart to re-acquire VPN permission), traits/trait objects, and a faster SSE codec.
- **The VPN-status channel = frb StreamSink.** A Rust function takes `StreamSink<T>`; Dart receives a `Stream<T>` to `listen()` — the canonical way to push continuous tunnel-state/handshake/bytes events from a long-lived Rust task (solves frb issue #347). This `Stream<T>` feeds a Riverpod `StreamProvider` directly.
- **UniFFI-Dart is NOT production-ready.** UniFFI's first-party targets are Kotlin/Swift/ Python/Ruby (3rd-party C#/Go); the community `uniffi-rs-dart` README states it is WIP and "should not be trusted." Use UniFFI only if the SAME Rust core must also serve native Kotlin/Swift/HarmonyOS-ArkTS surfaces (then run frb for Dart + UniFFI for Kotlin/Swift). Note Mullvad's own app uses a Rust daemon over gRPC/IPC rather than in-process FFI — an architecture worth weighing vs in-process frb.
- **Packaging:** Android `cdylib` → `lib<name>.so` per ABI via cargo-ndk in `jniLibs`; iOS/macOS `staticlib` (preferred for iOS) → `xcframework`; Linux/Windows `cdylib` loaded via Dart FFI `DynamicLibrary`. The Rust data path links into the OS VPN extension process (Android `VpnService`, iOS/macOS `NEPacketTunnelProvider`); frb bridges the UI/control side.
- **State:** Riverpod 3.x `StreamProvider.autoDispose` over the frb event stream. Note 3.0 semantics: `StreamProvider` **pauses its subscription when not listened**, and `overrideWithValue` was re-added (feed a fake VPN-state stream in tests, §11.4.27).
- **Aurora OS & HarmonyOS NEXT are second-tier forked targets** requiring bespoke per-platform VPN plugins; treat with honest SKIP-with-reason (§11.4.3) where the embedder/SIG port lacks a VPN/permission API — do NOT assume Android/iOS parity.

Version pins / API facts / constraints.

- **flutter_rust_bridge 2.12.0** (pub + docs.rs; prerelease 2.13.0-beta.2); a Flutter Favorite. **Pin the Dart-dep version == bridge-crate version.** Codegen step must run on every Rust API change.
- **flutter_riverpod 3.3.2** (Riverpod 3.0 released Sept 2025).
- **Aurora OS:** `gitlab.com/omprussia/flutter` (SDK fork + flutter-embedder + *_aurora community plugins); 2025-active (Mobius

2025 Flutter-CLI talk) but a separate fork. **HarmonyOS**
NEXT: gitee.com/openharmony-sig/flutter_flutter (dev branch),
engine on the Flutter OHOS branch — commonly **Flutter**
OHOS 3.22.x atop HarmonyOS SDK 5.0.0(12) / OpenHarmony
API 10-12, builds to .hap; needs an ArkTS VpnExtensionAbility
bridge to the Rust core.

Sources (accessed 2026-06-25):

- flutter_rust_bridge (v2.12.0, StreamSink) — https://pub.dev/packages/flutter_rust_bridge
 - frb V2 features — https://cjycode.com/flutter_rust_bridge/guides/miscellaneous/whats-new
 - frb repo — https://github.com/fzyzcjy/flutter_rust_bridge
 - frb issue #347 (long-lived Rust → continuous Dart stream) — https://github.com/fzyzcjy/flutter_rust_bridge/issues/347
 - frb crate 2.12.0 — https://docs.rs/crate/flutter_rust_bridge/latest
 - uniffi-rs — <https://github.com/mozilla/uniffi-rs>
 - uniffi-rs-dart (WIP, "should not be trusted") — <https://github.com/NiallBunting/uniffi-rs-dart>
 - UniFFI user guide — <https://mozilla.github.io/uniffi-rs/>
 - Aurora OS Flutter SDK fork — <https://gitlab.com/omprussia/flutter>
 - Aurora embedder — <https://gitlab.com/omprussia/flutter/flutter-embedder>
 - Flutter CLI for Aurora (Mobius 2025) — <https://mobiusconf.com/en/talks/26ff8152fa8e4038b85335c874c0b083/>
 - Flutter OHOS — <https://www.harmony-developers.com/p/flutter-app-development-hongmeng>
 - HarmonyOS Flutter setup — <https://dev.to/flfljh/setting-up-flutter-development-environment-for-harmonyos-hik>
 - Riverpod 3.0 — https://riverpod.dev/docs/whats_new
 - flutter_riverpod 3.3.2 — https://pub.dev/packages/flutter_riverpod
 - StreamProvider — <https://pub.dev/documentation/riverpod/latest/riverpod/StreamProvider-class.html>
-

A6. iOS / Android native tunnel core constraints

Key findings that shape the spec.

- **iOS NEPacketTunnelProvider memory limit is the binding cross-platform constraint.** Apple DTS (Quinn) measured **50 MiB** (iOS 15+, reconfirmed for iOS 16/17/18; history: ~15 MiB on iOS 11-14, raised to 50 MiB on iOS 15). **The limit is undocumented and MUST NOT be hardcoded**

(Quinn: "may well change ... the only way ... is to thoroughly test on a wide range of device and OS combinations"; may be lower on older devices). It applies to the **whole extension process** — your code AND every linked library (Rust staticlib, TLS, tun2socks) count.

- **Real-world risk:** a developer reported a **~15 MB jetsam kill on iOS 17.3.1** despite the documented 50 MiB (root cause PENDING_FORENSICS). Memory **spikes**, not steady state, are the killer — sing-box/WireGuard users see jetsam termination during **upload-heavy speedtests**. **Design rule:** target a steady-state working set **~12-15 MB** (not 50 MiB), bound packet-buffer pools, avoid per-flow state growth, size receive buffers to MTU+headroom. WireGuard-class (IP-level, no TCP connection tracking) fits naturally. No API to query/raise the limit — measure on device (Instruments / `phys_footprint` / Go `runtime.ReadMemStats` inside the NE).
- **Rust staticlib size optimization for iOS** (smaller code = fewer mapped pages = lower NE footprint): `[profile.release] opt-level="z" (try "s"), lto=true (fat), codegen-units=1, panic="abort", strip=true`. Biggest win: nightly `-Z build-std=std,panic_abort -Z build-std-features=panic_immediate_abort,optimize_for_size` (drops unwinding/gimli backtrace; ~43% reduction reported). Prefer staticlib .a for iOS (no dynamic-load entitlement friction; linker dead-strips). Watch monomorphization bloat and fat deps (regex/formatting/async runtimes).
- **Android VpnService:** `VpnService.Builder (addAddress/addRoute/addDnsServer/ setMtu/establish)` returns a `ParcelFileDescriptor` for the **Layer-3 TUN** (raw IP, single packet pipe, no per-socket model). Hand the raw fd to native via `ParcelFileDescriptor.detachFd()` (transfers ownership) and `read(2)/write(2)` directly (tun2socks pattern). **VpnService.protect(socket) is mandatory** before connect on every outbound transport socket (binds to the physical net, avoids the tunnel looping its own packets); high-throughput cores call `protectFromVpn (NetdClient.h)` directly via JNI to avoid per-socket upcalls. Android has **no hard iOS-style jetsam cap** — the severe memory constraint is iOS-only.
- **Synthesis:** design the shared Rust core to the iOS ~12-15 MB budget; same staticlib over JNI on Android (`detachFd` + native `protectFromVpn`); always measure on real devices under upload-heavy stress; never trust the documented iOS number.

Version pins / API facts / constraints.

- iOS NE limit **50 MiB measured, undocumented, possibly lower** (iOS 15-18); observed 15 MB kill on iOS 17.3.1 (UNCONFIRMED root cause).
- Android: `detachFd()` transfers fd ownership (native owns `close()`); `protect()` MUST precede socket connect; `protectFromVpn` is the native fast-path.

Sources (accessed 2026-06-25):

- Apple DevForums — packet tunnel = 50 MiB; "do not hard code" — <https://developer.apple.com/forums/thread/73148>
 - Apple DevForums — limit not documented, whole-process incl. libraries — <https://developer.apple.com/forums/thread/106377>
 - Apple DevForums — observed 15 MB jetsam kill on iOS 17.3.1 — <https://developer.apple.com/forums/thread/747474>
 - Apple DevForums — max received datagram size (read-buffer sizing) — <https://developer.apple.com/forums/thread/680486>
 - sing-box iOS memory crash during SpeedTest (WireGuard) — <https://github.com/SagerNet/sing-box/issues/3976>
 - Using Go in Apple network extensions (runtime.ReadMemStats) — <https://groups.google.com/g/traffic-obf/c/PksmyfHMUb4>
 - rustc codegen options — <https://doc.rust-lang.org/rustc/codegen-options/index.html>
 - Rust Performance Book build config — <https://nnethercote.github.io/perf-book/build-configuration.html>
 - Rust binary size: build-std / panic_immediate_abort / optimize_for_size — <https://markaicode.com/binary-size-optimization-techniques/>
 - bitdrift — optimizing Rust mobile SDK binary size — <https://blog.bitdrift.io/post/optimizing-rust-mobile-sdk-binary-size>
 - Reducing Rust binary size (mobile/iOS) — <https://ospfranco.com/rust-reduce-binary-size/>
 - Rust dylib/staticlib for iOS & Android — <https://ospfranco.com/complete-guide-to-dylibs-in-ios-and-android/>
 - VpnService (protect/establish/I-O) — <https://developer.android.com/reference/android/net/VpnService>
 - VpnService.Builder — <https://developer.android.com/reference/android/net/VpnService.Builder>
 - Passing the Android tun fd to native (detachFd) — <https://github.com/xjasonlyu/tun2socks/issues/123>
 - VpnService.protect via JNI (protectFromVpn/NetdClient) — <https://github.com/shadowsocks/shadowsocks-android/issues/2761>
-

A7. Go control-plane stack

Key findings that shape the spec.

- **Gin is the thin REST/JSON edge** (auth callbacks, health, admin, webhooks); keep the typed RPC surface on **Connect-RPC** and do NOT duplicate business logic. Connect handlers are plain `http.Handlers` — mount Connect + Gin on the **same `http.Server`** (route `/<package>.<Service>/` to Connect, everything else to Gin). Use `gin.New()` + explicit middleware (Recovery, structured logger, request-ID), not `gin.Default()`.

- **Connect-RPC: one handler speaks gRPC, gRPC-Web, and the Connect protocol** with no Envoy/translating proxy (browser/admin frontends talk binary gRPC-Web directly). **Server-streaming** is the VPN-relevant mode (push device/peer/relay-list updates, live tunnel status, account events). **Caveat:** a streaming response is always **HTTP 200** even on error — read the trailer status, not the HTTP code; bidi needs HTTP/2 end-to-end. Codegen via **buf** (buf.yaml + buf.gen.yaml, protoc-gen-go + protoc-gen-connect-go); buf lint/buf breaking in CI; pair with **protovalidate** (v1.0) via connectrpc/validate-go for declarative buf.validate field constraints.
- **sqlc = typed SQL, not an ORM.** sqlc.yaml version "2", engine: postgresql, sql_package: "pgx/v5", separate schema:/queries: dirs. With pgx/v5 generated code uses native pgtype (no database/sql Null* wrappers). overrides map domains/enums to Go types; generated Querier interfaces mock at unit level while integration tests hit real Postgres (§11.4.27). Run sqlc managed-DB verify in CI.
- **Postgres RLS for multi-tenant isolation** (the canonical 2025-26 pattern): app connects as a **non-owner, non-superuser role WITHOUT BYPASSRLS** (reserve BYPASSRLS for the migration role only; FORCE ROW LEVEL SECURITY if app owns a table); per-transaction tenant context via **SET LOCAL app.current_tenant** (plain SET leaks across pooled connections — the #1 isolation break); write **BOTH USING and WITH CHECK** (USING alone lets a tenant INSERT rows for another tenant); **index tenant_id as the leading composite column** (without it RLS is ~2 orders of magnitude slower). Wire as one WithTenantTx(ctx, accountID, fn) helper so no query runs without the context.
- **Redis Streams as durable work/event bus:** XREADGROUP > ... COUNT ... BLOCK + XACK; recover dead consumers with XAUTOCLAIM <min-idle-time> (production loop: XAUTOCLAIM-stalled then XREADGROUP-new); dead-letter poison messages via delivery-count cap → XADD <key>:dlq + XACK; trim with MAXLEN ~/XTRIM MINID. Redis 8.4+ folds reclaim into XREADGROUP ... CLAIM.
- **Migrations: goose is the safer default here** (hand-written reviewable CREATE POLICY ... USING ... WITH CHECK + FORCE ROW LEVEL SECURITY in the diff), with Atlas as the declarative drift-detection upgrade path (Atlas can import a goose project). Migration role uses BYPASSRLS; app role never does.

Version pins (all latest as of 2026-06-25).

- **Gin v1.12.0** (2025-02-28; v1.11.0 added HTTP/3 + swappable JSON codec).
- **connectrpc.com/connect v1.20.0** (min Go 1.25; supports the two most recent Go majors; protovalidate-go v1.0).
- **sqlc v1.31.1** (2026-04-22) + **jackc/pgx/v5 v5.10.0** (2026-06-03) via pgxpool.
- **go-redis/v9** (XAutoClaim/XReadGroup/XAck/XAdd MAXLEN).

- **goose v3.27.1** (2026-04-24); Atlas = declarative/auto-plan/drift alternative.

Sources (accessed 2026-06-25):

- Gin releases — <https://github.com/gin-gonic/gin/releases> · <https://gin-gonic.com/>
 - connect-go releases — <https://github.com/connectrpc/connect-go/releases> · <https://pkg.go.dev/connectrpc.com/connect>
 - Connect gRPC compat / streaming / getting-started / protocol — <https://connectrpc.com/docs/go/grpc-compatibility/> · <https://connectrpc.com/docs/go/streaming/> · <https://connectrpc.com/docs/go/getting-started/> · <https://connectrpc.com/docs/protocol/>
 - Connect vs gRPC — <https://buf.build/blog/connect-a-better-grpc>
 - protovalidate — <https://github.com/connectrpc/validate-go> · <https://github.com/bufbuild/protovalidate-go/releases>
 - sqlc — <https://github.com/sqlc-dev/sqlc/releases> · <https://docs.sqlc.dev/en/stable/guides/using-go-and-pgx.html> · <https://docs.sqlc.dev/en/latest/reference/datatypes.html> · <https://docs.sqlc.dev/en/latest/howto/overrides.html>
 - pgx v5.10.0 — <https://pkg.go.dev/github.com/jackc/pgx/v5>
 - Postgres RLS — <https://queryplane.com/blog/postgres-row-level-security-in-practice/> · <https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/> · <https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/> · <https://www.bytebase.com/reference/postgres/how-to/postgres-row-level-security/>
 - Redis Streams — <https://redis.io/tutorials/redis-backed-job-queue-for-background-workers/> · <https://redis.io/docs/latest/commands/xclaim/> · <https://oneuptime.com/blog/post/2026-01-21-redis-dead-letter-queue/view> · <https://oneuptime.com/blog/post/2026-03-31-redis-handle-consumer-failures-streams/view> · <https://redis.io/blog/single-shot-reliable-consumers-with-xreadgroup-claim-in-redis-84/> · <https://redis.io/docs/latest/develop/use-cases/streaming/>
 - Migrations — <https://github.com/pressly/goose/releases> · <https://volomn.com/blog/database-migration-using-atlas-and-goose> · <https://atlasgo.io/blog/2022/12/01/picking-database-migration-tool> · <https://codelit.io/blog/database-migration-tools-comparison> · <https://atlasgo.io/guides/migration-tools/goose-import>
-

A8. Deployment: rootless Podman quadlets → Compose → Kubernetes

Key findings that shape the spec.

- **Quadlet is core to Podman 5.x** (`podman quadlet list|print|install|rm`; unit types `.container/.pod/.build/.image/.artifact/.network/.volume/.kube`). A systemd generator emits real `.service` units. **Rootless is selected by file location, not a User= switch** — place units in a rootless search path (`~/.config/containers/systemd/`, etc.) and enable `loginctl enable-linger <user>` for boot-time start. "Quadlet units do not support running as a non-root user by defining the User, Group, or DynamicUser systemd options."
- **VPN-edge [Container] directives:** `DropCapability=ALL` then `AddCapability=NET_ADMIN NET_RAW`; `AddDevice=/dev/net/tun:/dev/net/tun` for userspace tunnels; `PublishPort=443:443/udp`; `Pod=helix.pod`; `ReadOnly=true + ReadOnlyTmpfs=true`; `SeccompProfile=<path>`; `Notify=true`. Pod-level `PublishPort/Network/Volume` apply pod-wide (members share the netns, reach each other on 127.0.0.1); the generator orders `*-pod.service` before members.
- **NET_ADMIN is necessary but NOT sufficient for tunnels:** WireGuard also needs `NET_RAW` (Docker enables it by default, Podman rootless does not), the **host WG kernel module loaded outside Podman** (kernels ≥ 5.6 ship it; a container can't modprobe rootlessly), and `/dev/net/tun` for userspace tunnels.
- **Rootless networking caveats (load-bearing):** **pasta** is the Podman 5.x / RHEL 9.5+ default backend (NAT-free, faster than `slirp4netns`) — **but a 5.8 throughput regression is reported, benchmark your build. Rootless Podman does NOT auto-install firewall/NAT rules** — provide host-side forwarding or in-netns masquerade for a VPN that routes client egress. MTU mis-sizing is a frequent "connected but nothing loads" cause. **Binding :443 rootlessly** = lower `net.ipv4.ip_unprivileged_port_start=443` (persist in `/etc/sysctl.d/`).
- **Cross-format bridge:** `podman kube play` runs K8s YAML; `podman kube generate` emits it from running containers.
- **Compose mapping:** `cap_drop:[ALL] + cap_add:[NET_ADMIN,NET_RAW]`, `devices:[/dev/net/tun:/dev/net/tun]`, `read_only:true + tmpfs:`, `security_opt:[seccomp=...]`, `ports:["443:443/udp"]`; no pod primitive (one project / shared network, service-DNS names).
- **Kubernetes mapping:** `securityContext (readOnlyRootFilesystem, allowPrivilegeEscalation:false, capabilities.drop:[ALL]/add:[NET_ADMIN,NET_RAW], seccompProfile.type: RuntimeDefault|Localhost)`. **K8s has no first-class /dev/net/tun** — privileged sidecar, device-plugin, or `hostPath` (each weakens isolation; the quadlet `AddDevice=` is

strictly simpler). Workloads: edge → Deployment (or DaemonSet for a host-interface bind); **Postgres** → **StatefulSet** (never Deployment); Redis → Deployment/StatefulSet; :443/udp Service needs explicit protocol: UDP; headless Services for DB/Redis; lock topology with a **default-deny NetworkPolicy** (CNI must enforce — Calico/Cilium).

Sources (accessed 2026-06-25):

- podman-systemd.unit — <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>
 - podman-quadlet — <https://docs.podman.io/en/latest/markdown/podman-quadlet.1.html>
 - rootless.md — <https://github.com/containers/podman/blob/main/rootless.md>
 - pasta 5.8 throughput regression — <https://github.com/containers/podman/issues/28219>
 - Red Hat Quadlet — <https://www.redhat.com/en/blog/quadlet-podman>
 - Podman Desktop Quadlet — <https://podman-desktop.io/blog/podman-quadlet>
 - Oracle Linux Quadlets — <https://docs.oracle.com/en/operating-systems/oracle-linux/podman/quadlets.html>
 - pasta networking — <https://docs.oracle.com/en/learn/ol-podman-pasta-networking/> · <https://sanj.dev/post/podman-pasta-vs-slrp4netns-networking/> · <https://github.com/eriksjolund/podman-networking-docs>
 - WireGuard in Podman — <https://www.procustodibus.com/blog/2022/10/wireguard-in-podman/> · <https://emar10.dev/posts/rootless-podman-wireguard/>
 - wg-easy rootless Podman — [https://github.com/wg-easy/wg-easy/wiki/Using-WireGuard-Easy-with-rootless-Podman-\(incl.-Kubernetes-yaml-file-generation\)](https://github.com/wg-easy/wg-easy/wiki/Using-WireGuard-Easy-with-rootless-Podman-(incl.-Kubernetes-yaml-file-generation))
 - Podman + WireGuard / privileged ports — <https://oneuptime.com/blog/post/2026-03-18-use-podman-containers-wireguard-vpn/view> · <https://oneuptime.com/blog/post/2026-03-18-bind-privileged-ports-rootless-podman/view> · <https://oneuptime.com/blog/post/2026-03-18-configure-ip-unprivileged-port-start-rootless-podman/view>
 - K8s securityContext — <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>
 - K8s capabilities drop-all/add — <https://oneuptime.com/blog/post/2026-02-09-capabilities-drop-all-add-specific/view>
 - Docker seccomp / capabilities / security context — <https://docs.docker.com/engine/security/seccomp/> · <https://lours.me/posts/compose-tip-029-container-capabilities/> · <https://oneuptime.com/blog/post/2026-01-30-docker-security-context/view> · <https://oneuptime.com/blog/post/2026-01-25-docker-container-capabilities/view>
-

A9. PKI / post-quantum WireGuard / NAT traversal

Key findings that shape the spec.

- **WireGuard's native identity = one Curve25519 keypair per device; the public key IS the identity.** Revocation is structurally trivial (remove the public key from the peer set → instant data-plane cutoff; no CRL/OCSP). WireGuard has **no built-in PKI** (no CA/expiry/CRL/OCSP/enrollment) — any cert lifecycle must be built *around* it.
- **Recommended PKI = SPIFFE/SPIRE-style short-lived mTLS over WG, with two device identities:** (1) a long-lived device attestation cert (optionally TPM/Secure-Enclave bound), and (2) a short-lived SVID (X.509/JWT, **default TTL 1 h, ≤60 min for hot paths**, agent auto-rotated). Revocation ≈ "stop renewing" (self-expiry within the TTL, no CRL distribution latency). **Generate the WG private key ON the device, never transmit it;** the control plane only ever receives the public key, binds it to the device cert/SVID, and distributes it to authorized peers. Belt-and-suspenders revocation = stop renewing the short-lived cert + remove the WG public key.
- **Post-quantum = PSK-injection (Mullvad/Rosenpass pattern), NOT a WG-handshake fork.** A separate PQ KEM exchange derives a PSK mixed into the WG handshake → "no less secure than WireGuard" + PQ HNDL protection. **Hybrid, not PQ-only** (classical + PQ KEM). Mullvad: **Classic McEliece + ML-KEM, default on desktop since 2025-01-09**, launched for iOS; embeds a Rosenpass server in its agent (since v0.25.4) auto-rotating PSKs; migrated Kyber→ML-KEM. Open-source build template: `mullvad/wgephemeralpeer`.
- **Standards anchor: FIPS 203 (ML-KEM) published 2024-08-13;** NSA CNSA 2.0 requires ML-KEM. Spec target: **hybrid X25519 + ML-KEM-768** (+ Classic McEliece for Mullvad-parity McEliece-strength HNDL), fed as a PSK into WG.
- **NAT traversal (Tailscale model):** direct P2P WG with a coordination server + **DERP relays** for setup/fallback; **STUN** discovers public IP:port; simultaneous send = **UDP hole punching** (~94% of NAT configs, "direct >90%"). **Symmetric ("hard") NAT** randomizes the source-port mapping → usually needs a relay (one-side-symmetric birthday-paradox probe: 256 ports, ~50% @2 s / 98% @20 s; dual-symmetric ~28 min → relay instead). Port-mapping helpers: UPnP-IGD, NAT-PMP, PCP.
- **DERP = Detoured Encrypted Routing Protocol over HTTP/HTTPS:** signaling channel + last-resort relay; **never decrypts** (dumb pipe for WG ciphertext, end-to-end preserved). **Peer Relays (Oct 2025)** let you designate your

OWN nodes as relays (relevant for self-hosted parity; Headscale supports custom DERP maps).

- **4via6** connects many overlapping-CIDR IPv4 networks without renumbering by mapping each site's IPv4 route into a unique IPv6 prefix (encodes **site ID 0-65535 (lower 16 bits)** + the IPv4 address); generate via tailscale debug via `<site-id> <ipv4-route>`; **ACLs must target the IPv6 CIDR**; requires Tailscale **v1.24+** on the subnet router.

Version pins / API facts / constraints.

- **Rosenpass v0.2.2 (2024-06-05)** — Rust, refreshes the WG PSK every **2 minutes**; KEMs in the tagged release = **Classic McEliece + Kyber-512** (NOT yet ML-KEM — verify upstream before relying on ML-KEM *in Rosenpass specifically*); uses liboqs; verification = symbolic ProVerif analysis, full crypto proof "in progress."
- SVID default TTL 1 h (≤ 60 min hot path); 4via6 needs Tailscale v1.24+.

Sources (accessed 2026-06-25):

- WireGuard identity/revocation — <https://contabo.com/blog/wireguard-vs-openvpn-a-deep-dive-protocol-comparison/> · <https://docs.vyos.io/en/latest/configuration/interfaces/wireguard.html>
- SPIFFE/SPIRE — <https://spiffe.io/docs/latest/spire-about/use-cases/> · <https://axelspire.com/business/device-identity-spiffe-workload/> · <https://debugg.ai/resources/goodbye-service-api-keys-spiffe-spire-workload-identity-zero-trust-mtls-kubernetes-multi-cloud-2025> · <https://www.spletzer.com/2025/03/zero-to-trusted-spiffe-and-spire-demystified/> · <https://petronellatech.com/blog/machine-identity-is-the-new-perimeter-mtls-spiffe-for-zero-trust/>
- Mullvad PQ — <https://mullvad.net/en/blog/quantum-resistant-tunnels-are-now-the-default-on-desktop> · <https://mullvad.net/en/blog/experimental-post-quantum-safe-vpn-tunnels> · <https://github.com/mullvad/wgephemeralpeer> · <https://www.techradar.com/pro/vpn/mullvad-launches-post-quantum-protection-for-iphones>
- Rosenpass — <https://github.com/rosenpass/rosenpass> · <https://news.ycombinator.com/item?id=34969760>
- PQ-WireGuard analysis — <https://thomwiggers.nl/publications/pq-wireguard/>
- FIPS 203 — <https://quantumxc.com/fips-203-validated-pqc/>
- NAT traversal / DERP — <https://tailscale.com/blog/how-nat-traversal-works> · <https://tailscale.com/blog/nat-traversal-improvements-pt-1> · <https://tailscale.com/docs/reference/derp-servers> · <https://www.sitepoint.com/tailscale-peer-relays-nat-traversal-derp/> · <https://tailscale.com/blog/peer-relays-international-networks> · <https://headscale.net/stable/ref/derp/>

- 4via6 — <https://tailscale.com/docs/features/subnet-routers/4via6-subnets> · <https://tailscale.com/blog/4via6-connectivity-to-edge-devices> · <https://schema.ai/technologies/tailscale/insights/4via6-requires-ipv6-destinations>
-

A10. DAITA / maybenot internals + VPN test-rig methodology

Key findings that shape the spec.

- **maybenot** is an open-source Rust framework for traffic-analysis defenses, an evolution of the Tor Circuit Padding Framework generalized to TLS/QUIC/**WireGuard**/Tor (paper: Pulls, WPES'23 / arXiv 2304.09510, doi 10.1145/3603216.3624953). A defense = **a list of probabilistic state machines + limits on padding and blocking**; each machine reacts to events (packet sent/received, timers) and triggers **padding** (inject cover packets) or **blocking** (delay outgoing traffic). "Limits" = the padding budget.
- **DAITA's three defenses (parity target):** (1) **Packet Size Normalization** — pad all packets to one constant size; (2) **Dummy Packet Injection** — unpredictable cover traffic; (3) **Traffic Pattern Distortion** — bidirectional cover traffic during bursts.
- **DAITA v1 client = four hardcoded machines:** inactivity padding (send padding if no data sent for a randomized **[1.5, 9.5] s**), a packet for every **3rd** received, a packet for every **5th** received, randomized-delay padding when idle. Relay-side per defense = three machines (NetFlow, Interspace-inspired padding, a unique anti-DF/RP machine). **DAITA v2** = no hardcoded client machines — the **relay selects from a DB and pushes machines + limits, ~half the dummy packets / half average bandwidth overhead** while retaining DF/RP protection; shipped incl. Linux and macOS.
- **Parity design takeaway:** implement (a) constant padding to one MTU-bounded size, (b) probabilistic dummy injection with a capped budget, (c) bidirectional burst cover, (d) the v2 server-pushed machine-selection model. **Reuse the maybenot crate + maybenot-machines** rather than reimplementing the engine.
- **Test-rig methodology (the spec's CI rig):**
 - **Linux netns + veth** build a client-ns ↔ relay-ns ↔ "internet-ns" topology with no real WAN — reproducible, CI-friendly.
 - **tc netem** injects latency/jitter/loss/dup/reorder/corruption inside a namespace; realism via delay-correlation and the **Gilbert-Elliott** bursty-loss model; combine with **HTB** to impair selected classes.

- **iperf3** measures TCP/UDP throughput/loss/jitter (control on TCP 5201); re-run under each netem profile for throughput-vs-impairment bars.
- **nftables**: fwmark-based **kill switch** (reject any egress not matching the WG fwmark → fail-closed), DNS-leak blocking (**drop UDP/TCP 53 on the physical interface**), and a "DPI simulation" (drop/mangle the VPN's wire signature to confirm obfuscation still connects).
- **Leak matrix**: DNS + WebRTC + IPv6 (IPv6 is the common gap — route through tunnel or block v6 egress); **kill-switch test** = bring the tunnel down mid-iperf3/curl, assert **zero packets** reach the upstream ns. Defense-in-depth DNS order: PostUp resolver → nftables drop of 53 → server-side resolver (Unbound/AdGuard Home) → server-side DNAT.

Version pins / API facts / constraints.

- maybenot Rust **MSRV 1.85**; crates: **maybenot core 2.2.2**, **maybenot-simulator 2.2.1**, **maybenot-ffi** (C FFI), **maybenot-machines 1.0.1** (machines-crate version is from a search snippet, **UNCONFIRMED** pending a direct crates.io read). 2.x line current. NetBird has an open issue to add DAITA via maybenot.
- **Negative findings (§11.4.6)**: DAITA v1 client constants ([1.5,9.5]s, 3rd/5th ratios) come from Tobias Pulls' (maybenot author) blog — authoritative for the design — while Mullvad's own blog states the mechanisms only qualitatively. Exact v2 padding-budget numbers and relay defense-DB size are not published beyond "half the dummy packets."

Sources (accessed 2026-06-25):

- maybenot — <https://github.com/maybenot-io/maybenot>
- maybenot paper — <https://arxiv.org/abs/2304.09510> · <https://dl.acm.org/doi/10.1145/3603216.3624953>
- maybenot crate versions — <https://libraries.io/cargo/maybenot> · <https://crates.io/crates/maybenot-simulator> · <https://crates.io/crates/maybenot>
- DAITA v1/v2 defenses — <https://pulls.name/blog/2025-03-27-daita-v1-and-v2-defenses/>
- DAITA — <https://mullvad.net/en/vpn/daita> · <https://mullvad.net/en/blog/daita-defense-against-ai-guided-traffic-analysis> · <https://mullvad.net/en/blog/introducing-defense-against-ai-guided-traffic-analysis-daita> · <https://mullvad.net/en/blog/defense-against-ai-guided-traffic-analysis-daita-now-available-on-linux-and-macos>
- CyberInsider DAITA v2 — <https://cyberinsider.com/mullvads-daita-v2-brings-stronger-resistance-to-ai-enhanced-vpn-traffic-analysis/>
- NetBird DAITA issue — <https://github.com/netbirdio/netbird/issues/2366>

- tc-netem — <https://man7.org/linux/man-pages/man8/tc-netem.8.html>
- netem RHEL9 / ip netns — <https://oneuptime.com/blog/post/2026-03-04-simulate-network-latency-packet-loss-tc-netem-rhel-9/view> · <https://oneuptime.com/blog/post/2026-03-02-how-to-use-ip-netns-for-network-namespaces-testing-on-ubuntu/view>
- network namespaces — https://girondi.net/post/network_namespaces/
- netem cookbook / bad networks — <https://srtlab.github.io/srt-cookbook/how-to-articles/using-netem-to-emulate-networks.html> · <https://samwho.dev/blog/emulating-bad-networks/>
- iperf3 — <https://www.redhat.com/en/blog/network-testing-iperf3> · <https://andrewbaker.ninja/2026/01/04/iperf3-the-engineers-swiss-army-knife-for-network-performance-testing/> · <https://www.cisconetsolutions.com/iperf-network-testing-and-troubleshooting-tool/>
- WireGuard kill switch / nftables leak prevention — <https://www.ivpn.net/knowledgebase/linux/linux-wireguard-kill-switch/> · <https://www.ivpn.net/knowledgebase/linux/linux-how-do-i-prevent-vpn-leaks-using-nftables-and-openvpn/>
- WireGuard DNS-leak prevention — <https://www.vpnsmith.com/en/blog/wireguard-dns-leak-prevention-2026>
- VPN leak testing (DNS/WebRTC/IPv6) — <https://vpn.how/en/pages/vpn-leak-testing-in-2026-step-by-step-guide-with-dns-webrtc-and-ipv6-checks.html> · <https://a.vpn.how/en/pages/ipv6-leaks-and-vpns-in-2026-how-to-seal-every-gap-without-losing-speed.html>